

LEARNING FROM DATA

Induction, Bias and the Choice of Model

*A self contained handbook, with worked arithmetic and illustrations,
written to be followed without any prior background.*

The Three Datasets Used Throughout

Everything in this handbook is demonstrated on three tiny datasets. They are small enough that every number can be checked by hand, and they are reused from beginning to end so that nothing has to be relearned.

Dataset A. Loan default, all features categorical.

ID	Employed	Credit history	Collateral	Default (target)
1	no	bad	no	yes
2	no	bad	yes	no
3	no	good	no	yes
4	yes	bad	no	yes
5	yes	good	yes	no
6	yes	good	yes	yes
7	yes	bad	yes	yes
8	no	good	yes	no

Note that instances 5 and 6 have **identical descriptions but different outcomes**. Real data behaves like this all the time, and it is the reason perfect prediction is usually impossible.

Dataset B. Exam outcome, both features continuous.

ID	Hours studied	Practice tests	Result (target)
1	2	1	fail
2	3	2	fail
3	4	1	fail
4	6	4	pass
5	7	5	pass
6	8	3	pass

Dataset C. Exam score, continuous target.

ID	Hours studied	Score (target)
1	1	2
2	2	4
3	3	5
4	4	8

What Learning from Data Means

The vocabulary, fixed once

- An **instance** is one row of the data. It is one example of the thing being predicted about. In Dataset A, one instance is one loan applicant.
- The **prediction subject** is what a row represents: an applicant, a patient, a transaction, a machine, a day.
- The **descriptive features** are the input columns. In Dataset A they are Employed, Credit history and Collateral.
- The **target feature** is the column to be predicted. In Dataset A it is Default.
- A **model** is a rule that maps descriptive features to the target feature.
- **Training** is the process of searching for that rule using the rows available.
- **Prediction** is applying the rule to a new row whose target is unknown.

In plain words

A model is a machine that takes in what you know about something and returns a guess about what you do not know. Training is the process of building that machine from examples where both parts were known.

Two kinds of target, two kinds of task

- If the target is **categorical**, meaning it takes one of a fixed set of labels, the task is **classification**. Dataset A (default yes or no) and Dataset B (pass or fail) are classification tasks.
- If the target is **continuous**, meaning it is a number on a scale, the task is **regression**. Dataset C (a score) is a regression task.

The kind of reasoning being used

Three kinds of reasoning are worth separating, because only one of them describes what a learning algorithm does.

- **Deduction** → from a general rule plus a fact, derive a guaranteed conclusion. *All applicants without collateral defaulted; this applicant has no collateral; therefore this applicant defaulted.* If the premises are true, the conclusion cannot be false.
- **Induction** → from a set of specific observations, infer a general rule. *In these eight rows, everybody without collateral defaulted; therefore applicants without collateral default.* The conclusion can be false even when every observation is true.
- **Abduction** → from an observation, infer the most plausible explanation. *This applicant defaulted; perhaps they lost their job.*

Inductive learning → learning that entails inducing a general rule from a set of specific instances.

Machine learning is inductive learning. Every consequence in this handbook follows from that single sentence.

The Two Hard Truths of Induction

First truth: an induced model is not guaranteed to be correct

A model learned by induction is **not guaranteed to be correct**.

It means: the general rule induced from a *sample* may not be true for all instances in the *population*.

Seeing it happen in Dataset A

Look only at instances 1, 3 and 4. All three lack collateral and all three defaulted. Induction gives the rule:

if Collateral = no then Default = yes

The rule is perfectly consistent with everything seen. It has zero error on those rows.

Now the population arrives. Instance 6 has collateral and still defaulted, so the rule already misses cases. Worse, nothing in the three rows examined rules out an applicant with no collateral who repays comfortably. The rule may simply be wrong, and no amount of staring at the three rows can reveal that.

- The gap between how well a model does on the rows it learned from and how well it does on new rows is called the **generalisation gap**.
- Training accuracy is therefore **not evidence**. A rule can fit the training rows perfectly and still be worthless.
- Confidence can only come from evidence gathered on data the model never saw during training. That is the entire reason for keeping a test set aside.

In plain words

No model comes with a proof. It comes with a track record. Treat every model the way you would treat a new employee: judge it on work you have not already shown it the answers to.

Second truth: learning cannot happen without bias

Learning **cannot occur** unless the learning process is **biased** in some way.

It means: the learning process must be told what *types of pattern* to look for in the data.

This sounds strange the first time it is heard, because bias is usually a word for something to be removed. The following count shows why bias is instead the thing that makes learning possible at all.

Counting how many rules could fit

Suppose there are m descriptive features, each with two possible values, and the target has two levels.

- The number of distinct possible descriptions is 2^m .
- Each description can be assigned either target level, so the number of possible rules, that is functions from descriptions to target levels, is

$$N = 2^{2^m}$$

Features m	Possible descriptions 2^m	Possible rules 2^{2^m}
3	8	256
4	16	65 536
6	64	about 1.8×10^{19}
10	1 024	about 1.8×10^{308}

Now suppose the training data covers k of the 2^m descriptions. Every rule that agrees with the data on those k descriptions is equally consistent with the evidence. The number of such rules is

$$2^{2^m - k}$$

and they **disagree with each other on every description not seen**.

With $m = 10$ and a generous 500 training descriptions, the number of rules that fit the data perfectly and yet give completely different answers on new cases is 2^{524} , a number with 159 digits.

- So a learner with no preferences has **no ground on which to choose** among all these equally consistent rules.
- It can memorise the training rows, but it cannot say anything about a new row.
- Therefore an unbiased learner is a useless learner. **Bias is not a defect of learning; it is the enabling condition of learning.**

In plain words

A student who is told only "here are 500 solved problems, now answer a new one" and who has no idea what kind of pattern to look for cannot do anything except look up the answer if it happens to be in the list. Telling the student what sort of thing to look for, for example "the answer usually depends on the ratio of the two numbers", is exactly what bias supplies.

Inductive Bias

The **inductive bias** of a learning algorithm comprises the set of assumptions that define

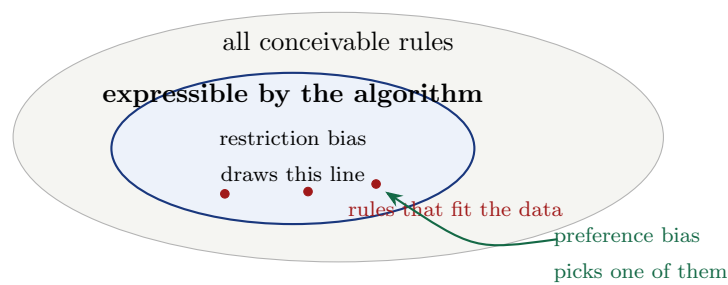
- (i) the **search space** the algorithm explores, and
- (ii) the **search process** it uses.

The two halves have names of their own, and separating them explains most algorithm behaviour.

- **Restriction bias** decides **which rules can even be expressed**. It shrinks the search space by fixing the shape of the model.
- **Preference bias** decides **which of the expressible rules is chosen** when several fit the data equally well. It orders the search space rather than shrinking it.

In plain words

Restriction bias decides what can be said. Preference bias decides what gets said first.



The inductive bias of the common algorithms, in one line each

- **Decision trees** → assume the target can be described by a hierarchy of tests on one feature at a time; prefer the most informative test first and prefer shorter trees.
- **Nearest neighbour** → assume that instances close together in the feature space share the same target; assume all features matter equally unless told otherwise.
- **Naive Bayes** → assumes the descriptive features are independent of one another once the target level is known.
- **Linear and logistic regression** → assume the target, or its log odds, is a weighted sum of the features; prefer the weights that minimise a stated error.
- **Support vector machines** → assume the classes can be separated in some transformed space; prefer the separating boundary that sits as far as possible from both classes.

Bias made visible on Dataset B

A straight line model can only draw one straight boundary through the hours and tests plane. If the true pass region were a circle in the middle of the plot, the straight line model would not merely fit it badly. It would be **incapable of expressing it**, no matter how much data or tuning were supplied. That is restriction bias doing its work.

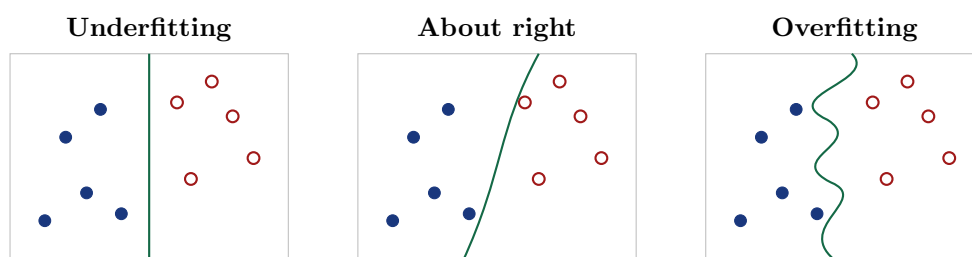
A nearest neighbour model has no such restriction and could trace the circle. It pays for that freedom by also tracing every stray mislabelled point.

Two senses of the word bias, kept apart

- **Inductive bias** is the set of assumptions an algorithm makes. Necessary, not a defect.
- **Statistical bias** is the systematic error of a model family, the counterpart of variance.

They are connected by a simple rule of thumb:

- Strong assumptions → small search space → low variance, high statistical bias → risk of **underfitting**, that is missing the real pattern.
- Weak assumptions → huge search space → low statistical bias, high variance → risk of **overfitting**, that is chasing the noise.



too rigid, real structure missed captures the pattern, ignores noise memorises every point, will not generalise

In plain words

Underfitting is a student who learned one crude slogan. Overfitting is a student who memorised the textbook word for word and cannot answer a question phrased differently. Both fail the exam, for opposite reasons.

Bias Enters at Every Decision, Not Only in the Algorithm

On top of the inductive bias encoded in the algorithm, the outcome of a project is biased in many other ways. Every one of the following questions has to be answered on every project, and the answer to each one introduces a specific bias.

- What is the prediction target?
- Which descriptive features will be included or excluded?
- How will missing values be handled?
- How will the features be normalised?
- How will continuous features be represented?
- What types of model will be created?
- How will the parameters of the learning algorithms be set?
- What evaluation process will be followed?
- What performance measures will be used?

One question, three answers, three different models

Take just the missing value question, on an income column where a fifth of the rows are blank.

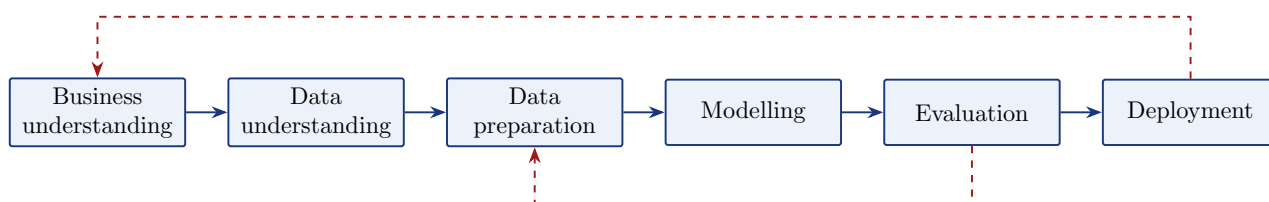
- **Delete those rows** → the model is trained only on people who disclose income, so it silently becomes a model about disclosers.
- **Fill with the average** → every blank becomes an average earner, so the model learns that non disclosure means ordinary income.
- **Fill with the average and add a flag column recording that it was missing** → the model can learn that non disclosure is itself informative, which it very often is.

Three defensible answers, three different datasets, three different models, and no theory that settles which is right. Only knowledge of the domain and experimentation can settle it.

- These questions are usually answered on the basis of **intuition, experience and experimentation**.
- This is what makes machine learning something of an art rather than strictly a science, and also what makes it rewarding work.
- Practical safeguard: keep a written **decision log** recording each answer and the reason for it. It is the only way a model can be reproduced, defended or debugged months later.

A lifecycle to keep the question set manageable

Taken all together the questions can seem overwhelming, which is why a structured process is used to manage a project through its lifecycle.



The dashed arrows are the feedback paths: evaluation regularly sends work back to data preparation, and deployment regularly reopens the business question.

The open questions belonging to each phase

Phase	Open questions
Business understanding	What is the organisational problem being addressed? In what ways could a prediction model address that problem? Do we have situational fluency, meaning enough domain grounding to talk with experts? What is the capacity of the organisation to use the output of a model? What data is available?
Data understanding	What is the prediction subject? What are the domain concepts? What is the target feature? Which descriptive features will be used?
Data preparation	Are there data quality issues? How will missing values be handled? How will features be normalised? Which features will be included?
Modelling	Which types of model will be used? How will the parameters of the learning algorithms be set? Has underfitting or overfitting occurred?
Evaluation	What evaluation process will be followed? Which performance measures will be used? Is the model fit for purpose?
Deployment	How will the model be integrated into the organisation? How will it continue to be evaluated after deployment?

- The lifecycle is **iterative**, not a straight line. Later phases feed back into earlier ones.
- The purpose of the project is to solve a **real problem**. The technical challenges of model building are fascinating and are a standing distraction from that purpose.

Four Equations Behind Four Ways of Learning

The commonly used prediction models can be organised around four approaches to learning, each imitating a way in which people themselves learn.

Approach	Central quantity	The human learning style it imitates
Information based	Shannon entropy	Learning by asking the most informative question next
Similarity based	Euclidean distance	Learning by analogy with remembered cases
Probability based	Bayes' Theorem	Learning by revising beliefs as evidence arrives
Error based	Sum of squared errors	Learning by repeatedly correcting mistakes

An understanding of these four quantities is a strong basis for the mathematical fundamentals of many areas of scientific modelling, not machine learning alone.

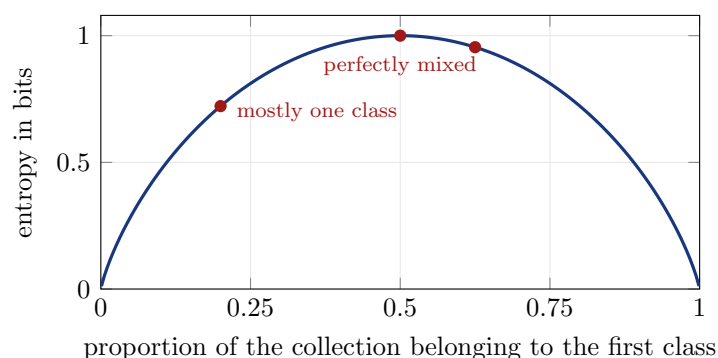
Equation one: entropy, or how to measure uncertainty

$$H(t) = - \sum_{i=1}^l P(t=i) \log_2(P(t=i))$$

Entropy is the negative of the sum, over all levels of a feature, of the probability of a level multiplied by the base two logarithm of that probability. It is measured in **bits**.

In plain words

Entropy answers one question: *how surprised will I be by the next value?* If a coin always lands heads, the answer is never, and entropy is zero. If it is fair, the answer is maximally, and entropy is one bit. Entropy is a number for how mixed up a collection is.



- Entropy is **zero** when every instance has the same target level. Such a collection is called **pure**.
- Entropy is **maximal** when the levels are equally common. Such a collection is maximally **impure**.
- For two levels the maximum is exactly one bit. For l equally likely levels it is $\log_2 l$.

From entropy to a decision tree: information gain

$$\text{rem}(f, \mathcal{D}) = \sum_{v \in \text{levels}(f)} \frac{|\mathcal{D}_{f=v}|}{|\mathcal{D}|} H(\mathcal{D}_{f=v}) \quad \text{IG}(f, \mathcal{D}) = H(\mathcal{D}) - \text{rem}(f, \mathcal{D})$$

The remainder is the entropy left over after splitting on a feature, weighted by how many instances fall into each branch. Information gain is how much uncertainty the split removed.

Building the first split of a decision tree on Dataset A

Step 1. Entropy before any split. Five of the eight applicants defaulted.

$$\begin{aligned} H(\mathcal{D}) &= -\frac{5}{8} \log_2 \frac{5}{8} - \frac{3}{8} \log_2 \frac{3}{8} \\ &= 0.625 \times 0.6781 + 0.375 \times 1.4150 \\ &= 0.4238 + 0.5306 = \mathbf{0.9544} \text{ bits} \end{aligned}$$

Step 2. Split on Collateral.

- Collateral = no gives instances 1, 3, 4. All three defaulted. Pure, so $H = 0$.
- Collateral = yes gives instances 2, 5, 6, 7, 8. Two defaulted, three did not.

$$\begin{aligned} H(\text{Collateral} = \text{yes}) &= -\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} = 0.4(1.3219) + 0.6(0.7370) = 0.9710 \\ \text{rem}(\text{Collateral}) &= \frac{3}{8}(0) + \frac{5}{8}(0.9710) = 0.6068 \\ \text{IG}(\text{Collateral}) &= 0.9544 - 0.6068 = \mathbf{0.3476} \end{aligned}$$

Step 3. Split on Employed.

- Employed = yes gives 4, 5, 6, 7: three defaults and one non default, so $H = 0.8113$.
- Employed = no gives 1, 2, 3, 8: two and two, so $H = 1$.

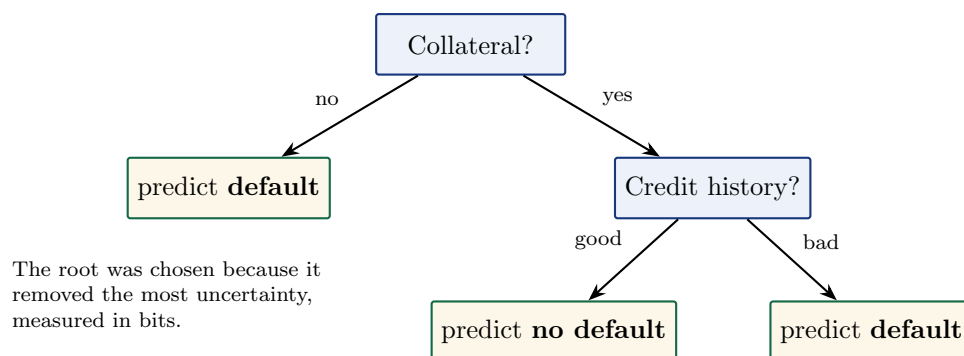
$$\text{rem}(\text{Employed}) = \frac{4}{8}(0.8113) + \frac{4}{8}(1) = 0.9056$$

$$\text{IG}(\text{Employed}) = 0.9544 - 0.9056 = \mathbf{0.0488}$$

Step 4. Split on Credit history. By the same arithmetic, good gives two and two and bad gives three and one, so

$$\text{rem}(\text{Credit}) = \frac{4}{8}(1) + \frac{4}{8}(0.8113) = 0.9056, \quad \text{IG}(\text{Credit}) = \mathbf{0.0488}$$

Step 5. Choose. Collateral gains 0.3476 bits, the others only 0.0488 bits each, so **Collateral becomes the root of the tree**. One branch is already pure and becomes a leaf predicting default. The other branch is split again by repeating the same calculation on the five instances it contains.



- Notice what the tree did to Employed and Credit history: it **ignored them at the root** because they carried little information. Feature selection is built into the algorithm itself.
- Notice also what it cannot do: every test looks at **one feature at a time**. That is the restriction bias of trees, and it is why their boundaries are stepped.

Equation two: distance, or learning by analogy

$$\text{dist}(\mathbf{a}, \mathbf{b}) = \sqrt{\sum_{i=1}^m (\mathbf{a}[i] - \mathbf{b}[i])^2}$$

The Euclidean distance between two instances is the square root of the sum of the squared differences between them, taken one feature at a time. It is the straight line distance in the feature space.

In plain words

This is the ordinary ruler distance, extended from a map to a table. If two rows have similar numbers in every column, they are close together, and the assumption is that close things behave alike.

Predicting a new student with Dataset B

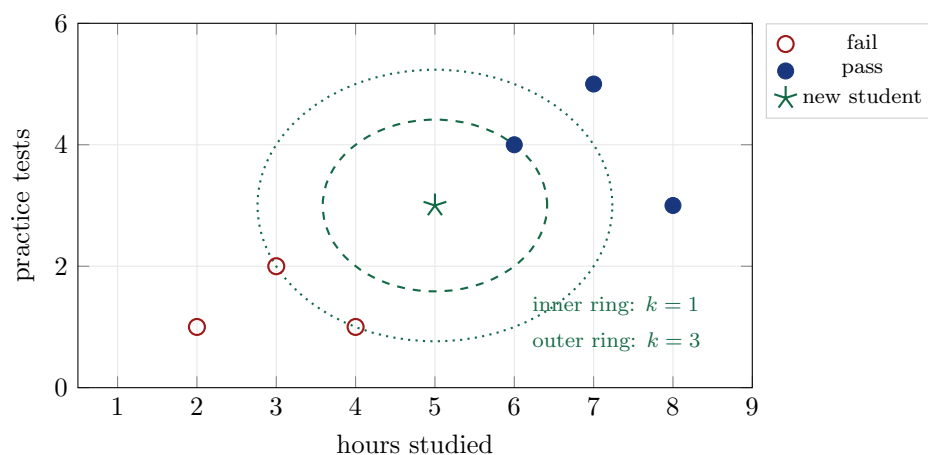
A new student has studied **5 hours** and taken **3 practice tests**. Distances to each known instance:

ID	Hours	Tests	Distance calculation	Result	
1	2	1	$\sqrt{(5-2)^2 + (3-1)^2} = \sqrt{9+4} = \sqrt{13}$	3.606	fail
2	3	2	$\sqrt{(5-3)^2 + (3-2)^2} = \sqrt{4+1} = \sqrt{5}$	2.236	fail
3	4	1	$\sqrt{(5-4)^2 + (3-1)^2} = \sqrt{1+4} = \sqrt{5}$	2.236	fail
4	6	4	$\sqrt{(5-6)^2 + (3-4)^2} = \sqrt{1+1} = \sqrt{2}$	1.414	pass
5	7	5	$\sqrt{(5-7)^2 + (3-5)^2} = \sqrt{4+4} = \sqrt{8}$	2.828	pass
6	8	3	$\sqrt{(5-8)^2 + (3-3)^2} = \sqrt{9+0} = \sqrt{9}$	3.000	pass

With $k = 1$: the single closest instance is 4 at distance 1.414, which passed. Prediction: **pass**.

With $k = 3$: the three closest are 4 (pass), 2 (fail) and 3 (fail). Majority vote gives two fails against one pass. Prediction: **fail**.

The same model, the same data, the same query, and two opposite answers. The value of k is not a detail. It is a direct control on how much the model smooths over local noise.



The one precaution that distance methods demand

Why unscaled features destroy a distance model

Suppose a dataset has Age in years and Income in rupees. Two applicants:

A: age 30, income 500 000

B: age 55, income 500 900

$$\begin{aligned}\text{dist}(A, B) &= \sqrt{(30 - 55)^2 + (500000 - 500900)^2} \\ &= \sqrt{625 + 810000} = \sqrt{810625} = 900.35\end{aligned}$$

The 25 year age difference contributed 625 out of 810 625, which is less than **0.08 percent** of the total. Income has effectively erased age from the model.

The fix. Rescale each feature to a common range before measuring distance, for example by range normalisation

$$a' = \frac{a - \min(a)}{\max(a) - \min(a)}$$

which places every feature between 0 and 1, or by standard scores, which express each value as a number of standard deviations from its mean.

- Euclidean distance is one member of a family. Setting the exponent to one gives **Manhattan distance**, the distance walked along a street grid. Other members suit other data: cosine similarity for text, Hamming distance for codes, Jaccard for sets.
- Distance methods do almost **no work during training** and a great deal of work at prediction time, since every query must be compared with the stored instances. This one fact drives most decisions about whether they can be deployed.

Equation three: Bayes' Theorem, or learning by revising belief

$$\underbrace{P(t|\mathbf{d})}_{\text{posterior}} = \frac{\overbrace{P(\mathbf{d}|t)}^{\text{likelihood}} \overbrace{P(t)}^{\text{prior}}}{\underbrace{P(\mathbf{d})}_{\text{evidence}}}$$

The probability of a target level given the evidence equals the probability of the evidence given that level, multiplied by the probability of the level before any evidence, divided by the overall probability of the evidence.

In plain words

Bayes' Theorem is the arithmetic of changing your mind. It takes what you believed before, combines it with how surprising the new evidence would be under each explanation, and returns what you should believe now.

The example that convinces everyone: a medical test

A disease affects **1 in 100** people. A test detects it in **90 percent** of people who have it, and wrongly reports it in **9 percent** of people who do not. Your test is positive. What is the probability that you have the disease?

Almost everyone answers about 90 percent. The arithmetic says otherwise. Imagine 10 000 people.

	Has disease	Does not	Total
Test positive	90	891	981
Test negative	10	9 009	9 019
Total	100	9 900	10 000

$$P(\text{disease} | \text{positive}) = \frac{90}{981} = 0.0917$$

or, straight from the formula,

$$P(D | +) = \frac{0.90 \times 0.01}{0.90 \times 0.01 + 0.09 \times 0.99} = \frac{0.009}{0.009 + 0.0891} = \frac{0.009}{0.0981} = \mathbf{9.2\%}$$

Why the intuition fails: the disease is rare, so the 9 percent of false alarms is drawn from a far larger group than the 90 percent of true detections. The prior dominates. A model that ignores the prior, in this case a model trained on a heavily imbalanced dataset without care, makes exactly this mistake.

From Bayes' Theorem to a working classifier

Applying the theorem directly needs $P(\mathbf{d} | t)$, the probability of an entire combination of feature values. With many features that combination is almost never observed often enough to estimate. The naive Bayes model resolves this with one assumption:

Conditional independence assumption $\rightarrow$ once the target level is known, the descriptive features are assumed independent of one another, so

$$P(\mathbf{d} | t) = \prod_{i=1}^m P(d_i | t)$$

and the prediction rule becomes

$$\hat{t} = \arg \max_{t \in \text{levels}} P(t) \prod_{i=1}^m P(d_i | t)$$

The denominator is the same for every level, so it can be ignored when comparing levels and restored only when a probability is needed.

Naive Bayes on Dataset A, including what happens when a count is zero

Query: an applicant who is **employed**, has **good credit**, and offers **no collateral**. This combination does not appear anywhere in the eight rows, so no lookup is possible and the model must genuinely generalise.

Priors. Five defaults out of eight, three non defaults out of eight:

$$P(\text{yes}) = \frac{5}{8} = 0.625, \quad P(\text{no}) = \frac{3}{8} = 0.375$$

Likelihoods for default = yes (instances 1, 3, 4, 6, 7):

$$P(\text{Employed} = \text{yes} | \text{yes}) = \frac{3}{5}, \quad P(\text{Credit} = \text{good} | \text{yes}) = \frac{2}{5}, \quad P(\text{Collateral} = \text{no} | \text{yes}) = \frac{3}{5}$$

$$\text{score}(\text{yes}) = 0.625 \times 0.6 \times 0.4 \times 0.6 = 0.0900$$

Likelihoods for default = no (instances 2, 5, 8):

$$P(\text{Employed} = \text{yes} | \text{no}) = \frac{1}{3}, \quad P(\text{Credit} = \text{good} | \text{no}) = \frac{2}{3}, \quad P(\text{Collateral} = \text{no} | \text{no}) = \frac{0}{3} = 0$$

$$\text{score}(\text{no}) = 0.375 \times 0.3333 \times 0.6667 \times 0 = \mathbf{0}$$

The zero problem. Not one non defaulter in this sample happened to lack collateral. A single missing combination has driven the whole product to zero, which claims it is *impossible* for such an applicant to repay. That is a statement the data never justified.

Laplace smoothing. Add one to every count and add the number of possible values to every denominator, so that nothing is ever assigned probability zero:

$$P(f = v | t) = \frac{\text{count}(f = v, t) + 1}{\text{count}(t) + |\text{values}(f)|}$$

Recomputing with two values per feature:

$$\text{score}(\text{yes}) = 0.625 \times \frac{4}{7} \times \frac{3}{7} \times \frac{4}{7} = 0.625 \times 0.5714 \times 0.4286 \times 0.5714 = 0.0875$$

$$\text{score}(\text{no}) = 0.375 \times \frac{2}{5} \times \frac{3}{5} \times \frac{1}{5} = 0.375 \times 0.4 \times 0.6 \times 0.2 = 0.0180$$

Normalising so the two add to one:

$$P(\text{default} | \text{query}) = \frac{0.0875}{0.0875 + 0.0180} = \mathbf{0.829}$$

Prediction: default, with about 83 percent confidence. The smoothed model is now able to answer a question about a combination it never saw, which is exactly what generalisation means.

- The independence assumption is usually **false**. Income and job title are plainly related. Naive Bayes works well anyway in many domains, because the ranking of the levels often survives even when the probabilities themselves are distorted.
- This is inductive bias in its purest form: a strong assumption, known to be imperfect, adopted because it makes learning possible from a small amount of data.

Equation four: squared error, or learning by correcting mistakes

$$L_2 = \frac{1}{2} \sum_{i=1}^n (t_i - \mathbb{M}(\mathbf{d}_i))^2$$

The sum of squared errors adds up, over every training instance, the square of the gap between the true target value and the value the model predicts. The one half is a convenience that cancels when the expression is differentiated.

In plain words

This is a score for how wrong a model is overall. Squaring does two things: it stops positive and negative mistakes from cancelling each other out, and it punishes one large mistake far more than several small ones.

Scoring three candidate models on Dataset C

The model is a straight line $\mathbb{M}(x) = w_0 + w_1x$, where x is hours studied.

Candidate A: $w_0 = 1$, $w_1 = 1.5$

x	true t	prediction	error	squared error
1	2	2.5	-0.5	0.25
2	4	4.0	0.0	0.00
3	5	5.5	-0.5	0.25
4	8	7.0	1.0	1.00
sum of squared errors				1.50

Candidate B: $w_0 = 0$, $w_1 = 2$

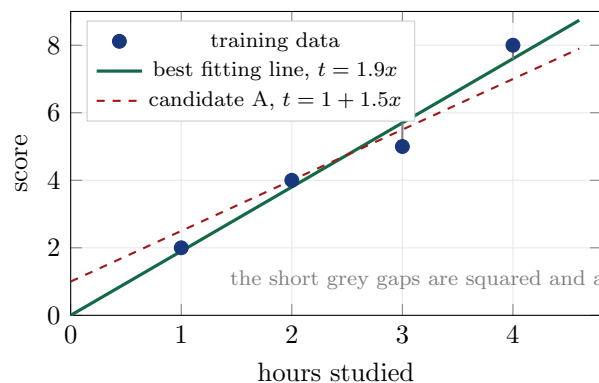
x	true t	prediction	error	squared error
1	2	2	0	0.00
2	4	4	0	0.00
3	5	6	-1	1.00
4	8	8	0	0.00
sum of squared errors				1.00

Candidate C: the best possible line through the origin. For a model $\mathbb{M}(x) = w_1x$ the optimum can be found exactly:

$$w_1 = \frac{\sum x_i t_i}{\sum x_i^2} = \frac{(1)(2) + (2)(4) + (3)(5) + (4)(8)}{1 + 4 + 9 + 16} = \frac{2 + 8 + 15 + 32}{30} = \frac{57}{30} = 1.9$$

x	true t	prediction	error	squared error
1	2	1.9	0.1	0.01
2	4	3.8	0.2	0.04
3	5	5.7	-0.7	0.49
4	8	7.6	0.4	0.16
sum of squared errors				0.70

The tables above report the plain sum of squares. The one half in the definition simply halves every height and does not move the minimum. Candidate C wins, and no other line through the origin can beat it. Note that its error is not zero. **The best model is not the one with no error; it is the one with the least error that the chosen family of models can achieve.**



Finding the weights when no formula exists: gradient descent

For most model families there is no neat closed formula for the best weights, so they are found by walking downhill on the error surface. The rule is:

$$w_j \leftarrow w_j - \alpha \frac{\partial L_2}{\partial w_j} \quad \text{where for } \mathbb{M}(x) = w_1 x \text{ we have } \frac{\partial L_2}{\partial w_1} = - \sum_i (t_i - w_1 x_i) x_i$$

The step size α is called the **learning rate**.

Two steps of gradient descent by hand, Dataset C

Start at $w_1 = 1$ with learning rate $\alpha = 0.02$.

Step 1. Predictions are 1, 2, 3, 4; errors are 1, 2, 2, 4.

$$\frac{\partial L_2}{\partial w_1} = -[(1)(1) + (2)(2) + (2)(3) + (4)(4)] = -(1 + 4 + 6 + 16) = -27$$

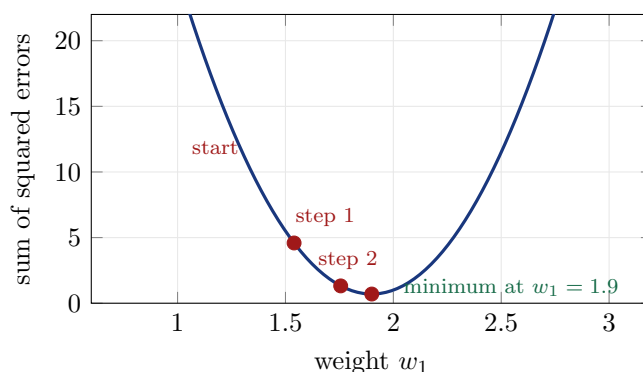
$$w_1 \leftarrow 1 - 0.02(-27) = 1 + 0.54 = \mathbf{1.54}$$

Step 2. Predictions are 1.54, 3.08, 4.62, 6.16; errors are 0.46, 0.92, 0.38, 1.84.

$$\frac{\partial L_2}{\partial w_1} = -[0.46 + 1.84 + 1.14 + 7.36] = -10.8$$

$$w_1 \leftarrow 1.54 + 0.216 = \mathbf{1.756}$$

The weight is walking towards the exact answer of 1.9, and the steps are getting smaller as the slope flattens. That shrinking step is automatic: near the bottom of the valley the gradient is small, so the algorithm slows down of its own accord.



- If α is **too small**, the walk takes forever. If it is **too large**, the steps overshoot the valley floor and the error can grow instead of shrink.
- The surface above is a simple bowl with one lowest point. For more complex models the surface has many valleys, and where the walk ends can depend on where it started.

Parametric and Nonparametric Models

The four way grouping above is based on the human learning style each algorithm imitates. It is not the only useful distinction, and the others give real insight into which algorithm suits a given situation.

The distinction is not absolute. It describes whether the size of the representation used to define a model is determined **solely by the number of features**, or is **also affected by the number of instances** in the dataset.

- In a parametric model the number of parameters is **independent of the number of instances**.
- In a nonparametric model the number of parameters **increases as the number of instances increases**.

Counting the parameters, so the definition becomes concrete

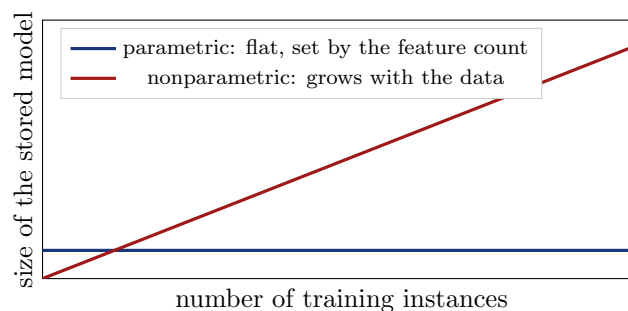
Linear regression on Dataset C. The model is $w_0 + w_1x$. That is **two numbers**. Add ten million more students to the training data and the model is still two numbers. With m features it is $m + 1$ numbers, whatever n may be. **Parametric.**

Naive Bayes on Dataset A. Three features with two values each, and two target levels, require $3 \times 2 \times 2 = 12$ conditional probabilities plus 2 priors, so **14 numbers**. Add a million rows and it is still 14 numbers, only estimated more precisely. **Parametric.**

Nearest neighbour on Dataset B. The model *is* the six stored instances. Add a million rows and the model is a million rows. **Nonparametric.**

Decision tree on Dataset A. No fixed set of parameters is assumed before training. The branching and depth follow the complexity of the data, so a different dataset produces a tree of a different size and shape. **Nonparametric.**

Support vector machine. It keeps some instances, the ones nearest the boundary, as part of its representation. How many it keeps depends on the data, so the size can change as instances are added. **Nonparametric**, although in practice relatively few instances are kept, which is why it scales far better than most nonparametric methods.



What follows from the distinction

- Parametric models make **stronger assumptions** about the underlying distributions. Assuming the relationship is a straight line is a very strong claim about a domain.
- Nonparametric models are **more flexible** but can struggle with large datasets. A nearest neighbour model can trace a broken, discontinuous boundary that no straight line could express, but it runs into time and memory problems as instances accumulate.
- On a **small dataset** a parametric model may perform better, because its strong assumptions, *if they happen to be correct*, protect it from overfitting.
- As the dataset **grows**, particularly when the boundary between classes is complicated, it makes more sense to let the data speak more directly, so nonparametric methods become attractive.
- The computational cost of nonparametric models on large data cannot be ignored. Support vector machines largely escape this problem, which is why they are often a good choice in **complex**

domains with plenty of data.**In plain words**

A parametric model is a summary. A nonparametric model is a filing cabinet. A summary is quick to carry and quick to consult but throws detail away. A filing cabinet keeps every detail and gets heavier every day.

Generative and Discriminative Models

A model is **generative** if it can be used to generate data with the same characteristics as the dataset it was built from. To do that it must **learn or encode the distribution of the data belonging to each class**.

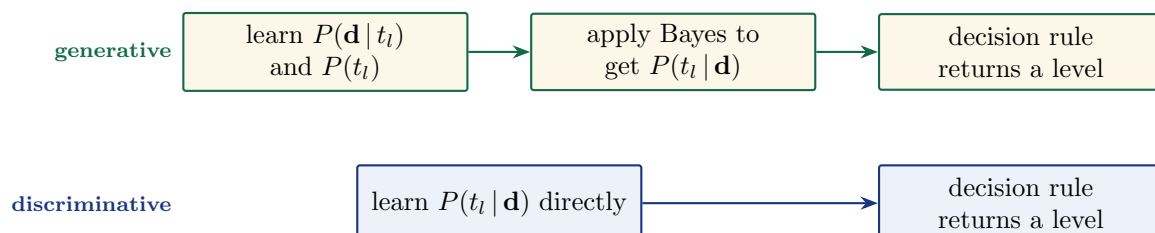
A model is **discriminative** if it learns the **boundary between the classes** rather than the characteristics of the distribution of each class.

In plain words

Ask the model a strange question: *make up a new, realistic example of a defaulter*. A generative model can do it, because it knows what defaulters look like. A discriminative model cannot, because all it ever learned was where to draw the line between the two groups. Both can classify; only one can imagine.

What each one actually learns, step by step

Writing $\mathbf{d}$ for a vector of descriptive feature values and t_l for a target level:

**The same prediction made both ways on Dataset A**

The generative route is exactly the naive Bayes calculation done earlier. It first learned what employed status, credit history and collateral look like *among defaulters*, and separately *among non defaulters*, together with how common each group is. Only then did Bayes' Theorem turn those descriptions into a prediction.

The discriminative route would instead fit something like

$$P(\text{default} | \mathbf{d}) = \frac{1}{1 + e^{-(w_0 + w_1 d_1 + w_2 d_2 + w_3 d_3)}}$$

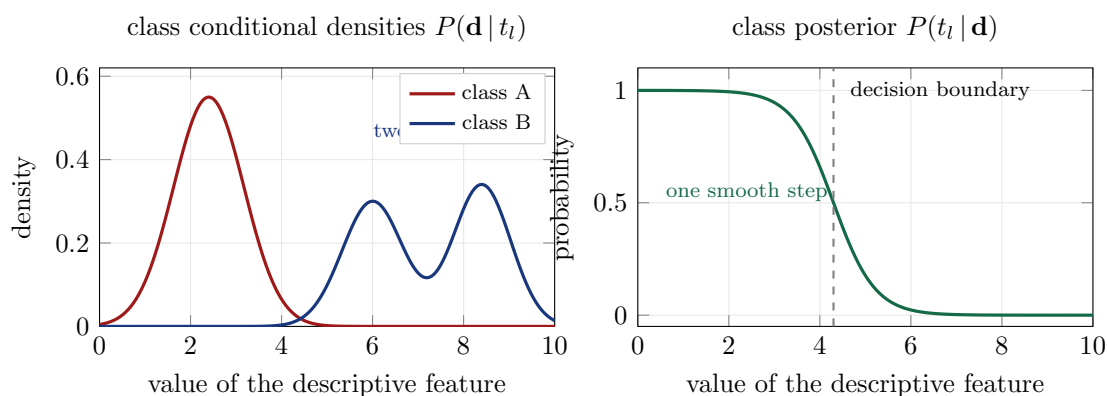
and adjust the weights until the predicted probabilities match the observed outcomes as closely as possible. At no point does it form any picture of what a defaulter looks like. It only learns where the dividing line sits.

Which of the two is harder, and why it matters

- The class conditional densities $P(\mathbf{d} | t_l)$ can be very complex compared with the class posteriors $P(t_l | \mathbf{d})$.

- Consequently a generative model tries to **solve a harder problem** than the prediction task strictly requires.
- This is worse when there are many features, because as the dimensionality grows, more and more data is needed to estimate a distribution well.
- Therefore, in complex domains discriminative models are likely to be more accurate.

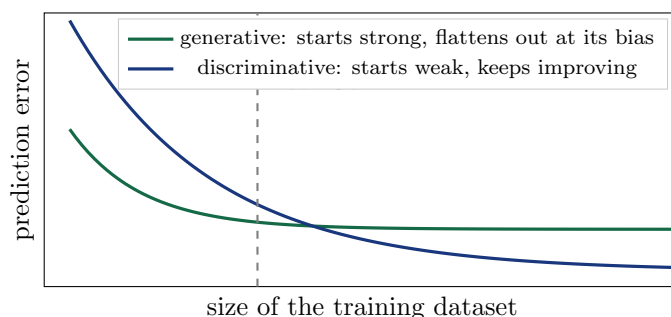
The picture below is the whole argument in one image. On the left, the distribution of a single feature within each class, one of them lumpy and awkward. On the right, the probability of the class given the feature, computed from those same distributions.



- The second hump in class B changes the left hand picture a great deal and the right hand picture hardly at all, because far out to the right class B already wins by a wide margin.
- A generative model has to get that entire lumpy shape right. A discriminative model only has to find the place where the two classes change rank.

But the argument does not end there

- Generative models carry **higher bias**: they make more assumptions about the form of the distribution being learned, including independence assumptions among the features.
- That sounds like another disadvantage. However, in domains where there is **good prior knowledge** about how the features relate, that knowledge can be encoded into a generative model.
- The encoded structure biases the model in a way that helps it **avoid overfitting**.
- So a generative model may **outperform** a discriminative one when trained on a **small dataset with good prior knowledge**.
- As training data grows, the bias imposed on the generative model eventually becomes larger than the error of the trained model. Past this **tipping point in dataset size**, the discriminative model will win.



- The debate extends beyond accuracy to the handling of missing data, unlabelled data and feature preprocessing.
- **Practical conclusion:** the right choice is context dependent, and evaluating a range of different types of model is the safest option.

The combined picture for the standard models

Model	Parametric status	Learning style	Why
k Nearest Neighbour	Nonparametric	Generative	keeps the instances, so it holds a picture of each class
Decision Trees	Nonparametric	Discriminative	partitions the space into class regions, which forms a boundary
Bagging and Boosting	Parametric *	Discriminative	aggregates boundary learners
Naive Bayes	Parametric	Generative	learns each class distribution, then applies Bayes
Bayesian Network	Parametric	Generative	encodes a full joint distribution and can be run to produce data
Linear Regression	Parametric	Discriminative	fits a target surface directly
Logistic Regression	Parametric	Discriminative	fits the class probability directly
Support Vector Machine	Nonparametric	Discriminative	finds the maximum margin boundary

* Although the individual models inside an ensemble may themselves be nonparametric, for example when decision trees are used, the ensemble model itself is considered parametric.

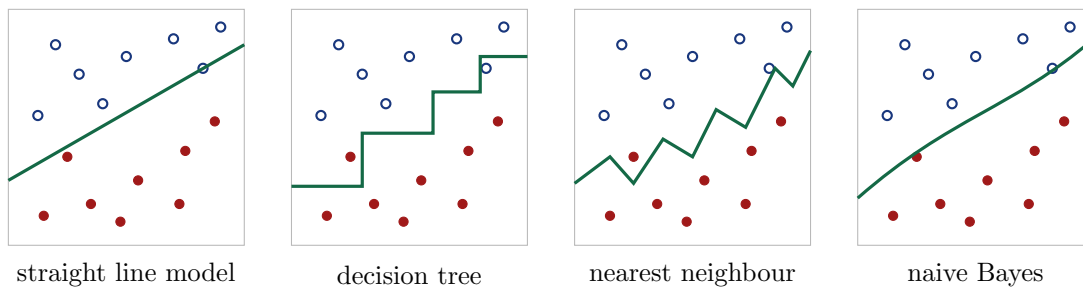
- **Qualification one.** These labels are **general case** statements. In principle any model can be trained in either a generative or a discriminative manner. Some simply lend themselves to one style, and the table reflects that tendency.
- **Qualification two.** A generative model can equivalently be described as learning the **joint distribution** $P(\mathbf{d}, t_l)$ and computing the required posteriors from it.

No Free Lunch, and the Shape of Decision Boundaries

The No Free Lunch Theorem → there is no single approach that outperforms all others across all possible problems.

It means: each algorithm encodes a distinct set of assumptions, namely its inductive bias, and a set of assumptions that is appropriate in one domain may be quite wrong in another.

The assumptions are not hidden. They are visible in the shape of the boundary each algorithm draws. The gallery below uses the same underlying truth in each column and the same four algorithms in each row, so the differences are entirely due to the algorithms.



Two things this gallery shows.

1. The boundary each algorithm learns is characteristic of that algorithm.

- Decision tree boundaries have a distinctive **stepped** appearance, because each test splits on one feature at a time, so every boundary segment must run parallel to an axis.
- Nearest neighbour boundaries are noticeably **jagged**, because each part of the boundary is decided locally by whichever instances happen to be nearby.
- Naive Bayes with normal distributions produces a **smooth curve**, whose shape comes from the assumed distributional form.
- A simple linear model produces exactly one **straight line**, because it can express nothing else.
- These appearances are direct expressions of the representation used and the inductive bias encoded in the algorithm.

2. Which model represents the truth best changes with the dataset. If the true boundary is a straight line, the linear model wins and the jagged one merely adds noise. If the true boundary is a set of rectangular thresholds, the tree wins and the straight line cannot get close. If the true boundary is an irregular island, nearest neighbour wins and both of the others fail.

In plain words

Every algorithm is a pair of spectacles with a fixed prescription. Some prescriptions bring some worlds into focus and blur others. There is no pair of spectacles that is sharpest for every world, and that, in one sentence, is No Free Lunch.

What follows for real projects

- Real datasets are far more complex than any two feature illustration, and cannot be plotted and inspected in this way.
- In general it is **not a good idea** to select one approach at the beginning of a project and use only that.
- It is better to choose a number of different approaches and run experiments to evaluate which is best for the particular project.
- That still requires an **initial shortlist**, and two questions guide it:
 1. Does the approach match the **requirements of the project**?
 2. Is the approach suitable for the **type of prediction** being made and the **types of descriptive features** available?

Matching an Approach to the Project

Accuracy, and why it is only the first question

- In many cases the primary requirement is an **accurate** model, and accuracy is related to the power of an algorithm to capture the interaction between the descriptive features and the target.
- Broad empirical comparisons across many domains report that, **on average, ensembles and support vector machines are among the most accurate models.**
- The consistent and more instructive finding is that **in some domains these powerful models perform quite poorly**, while models that are weak elsewhere achieve the best results.
- **Conclusion:** no approach is universally best, and **experimentation with different approaches is the best way to ensure that an accurate model is built.**

Three requirements that regularly outrank accuracy

Prediction speed → *how quickly can the model score a new instance?*

- Logistic regression is **very fast**. Scoring means evaluating one equation and comparing the result with a threshold.
- Nearest neighbour is **very slow**. Every prediction requires comparing the query against a typically large training set.

Putting numbers on it

Let n be the number of stored instances and m the number of features.

Model	Work per prediction	With $n = 1\,000\,000$, $m = 20$
Logistic regression	about m multiplications	20 operations
Decision tree	about the depth of the tree	roughly 10 to 20 comparisons
Naive Bayes	about m lookups and multiplications	20 operations
Nearest neighbour	about $n \times m$ operations	20 000 000 operations

A real time card fraud system may need thousands of predictions per second. At twenty million operations each, a nearest neighbour model is a million times more expensive per decision than the regression model. No reasonable amount of hardware closes a gap of that size.

Capacity for retraining → *how easily can the model be adapted once it goes stale?*

- Model performance can be monitored to flag **concept drift** and to indicate that a model has gone **stale**, meaning the world has changed underneath it.
- **Easy to adapt:** naive Bayes, where new instances simply update counts, and nearest neighbour, where new instances are appended.
- **Hard to adapt:** decision trees and regression models. In practice the only option is to **discard the model and train a new one** on an updated dataset.

Adding one instance to the naive Bayes model of Dataset A

Suppose a ninth applicant arrives: employed, good credit, collateral offered, and did not default. Only counts change.

- Non defaults rise from 3 to 4, so $P(\text{no})$ becomes $\frac{4}{9}$ and $P(\text{yes})$ becomes $\frac{5}{9}$.
- Among non defaulters, employed rises from 1 to 2 out of 4, good credit from 2 to 3 out of

4, collateral from 3 to 4 out of 4.

The update took a few additions. Retraining a decision tree on the same nine rows means recomputing every entropy and every information gain from the beginning, and the resulting tree may have a different root.

Interpretability → *can the prediction be explained and justified?*

- Businesses frequently will not simply accept predictions and act on them. They require the predictions to be **explained and justified**.
- **Very easily interpreted:** decision trees, which give a readable rule path, and linear regression, which gives one signed weight per feature.
- **Almost entirely uninterpretable:** support vector machines and ensembles, which for this reason are often called a **black box**.

The same refusal, explained two ways

From the decision tree built earlier:

"The application was declined because no collateral was offered, and in the available records every applicant without collateral defaulted."

That sentence can be printed on a letter, audited by a regulator, and challenged by the customer.

From an ensemble of five hundred trees:

"The application was declined because the aggregated score across five hundred models was 0.71, which is above the cutoff of 0.5."

That sentence explains nothing, cannot be checked by the customer, and in many regulated settings cannot lawfully be the only justification given.

Other requirements worth checking before committing

- **Training cost** → can the model be retrained inside the operational window, on the hardware available?
- **Memory at prediction time** → an instance based model must ship the dataset with it, which decides whether it can run on a phone or a small device.
- **Robustness** → how badly is the model affected by noise, outliers and irrelevant features?
- **Data privacy** → a model that stores instances effectively stores personal records, which is a serious constraint wherever privacy rules apply.
- **Fairness** → can performance be measured separately for different groups, and can disparities be corrected?
- **Calibration** → are the probabilities trustworthy as probabilities, and not merely as a ranking?
- **Cost of the two kinds of error** → a false alarm and a missed case rarely cost the same, and the threshold should reflect the difference.

Why the error costs decide the threshold

Take a screening model with these costs: missing a genuine fraud costs 50 000, and wrongly flagging an honest transaction costs 100 in review time. If the model outputs a probability p

of fraud, flagging is worth it when

$$p \times 50\,000 > (1 - p) \times 100 \quad \implies \quad p > \frac{100}{50\,100} = 0.002$$

The sensible cutoff is two tenths of one percent, not the default fifty percent. A model evaluated at the default threshold would look far worse than it truly is, and a model deployed at the default threshold would miss almost every fraud.

The summary judgement

- Ensembles, support vector machines and Bayesian networks are in general **more powerful** than the simpler approaches.
- They are also **more complex, longer to train, more heavily biased in their assumptions, and harder to interpret.**
- **Often speed, retraining capacity and interpretability drive the final selection more strongly than prediction accuracy does.**

Matching an Approach to the Data

The ground rule

- Almost every approach **can be made to work** with both continuous and categorical descriptive and target features.
- Certain approaches are nevertheless a **more natural fit** for some kinds of data, meaning less encoding, less distortion and less tuning is needed before they work.

Start with the target feature

- **Continuous target.**
 - Models trained by reducing the sum of squared errors, such as linear regression, are the **most natural fit**.
 - Information based and probability based approaches are the **least well suited** here.
- **Categorical target.**
 - Information based and probability based approaches are likely to work very well.
 - Error based approaches become **overly complicated once the target has more than two levels**, because the problem has to be decomposed into several binary problems.

Why more than two levels complicates an error based model

A target with three levels, low, medium and high, cannot be handled by a single line with a single threshold. The usual repairs are:

- **one against the rest** → train three separate models, low versus not low, medium versus not medium, high versus not high, then reconcile the three answers when two of them both say yes;
- **one against one** → train a model for each pair, which for l levels means $\frac{l(l-1)}{2}$ models, so 3 models for 3 levels but 45 for 10.

An information based or probability based model needs none of this. A tree simply puts three

kinds of leaf in the tree, and naive Bayes simply computes three scores and takes the largest.

Then look at the descriptive features

- **All descriptive features continuous.**
 - A **similarity based** approach is a natural fit, especially with a categorical target.
 - If the target is also continuous, an **error based** model is generally preferred.
 - When there are **many** continuous features, probability based and information based models become complicated, since densities must be estimated or values discretised.
- **All descriptive features categorical.**
 - **Information based** and **probability based** models are appropriate and comfortable.
 - **Error based** models are less suitable, because categorical features must be converted into sets of binary features, which causes an **increase in dimensionality**.
- **A mixture of both.** In many datasets both types appear. The most naturally suited approaches are those best suited to **the majority feature type**.

How a single categorical column inflates the dimensionality

A column holding a postal region with 400 possible values must be presented to a regression model as 400 binary columns, of which exactly one is set to one and the rest to zero. One column has become 400. Add a product code with 1 200 values and the dataset has 1 600 columns before any real measurement is included.

A decision tree simply asks whether the region is in a particular set of values, and naive Bayes simply keeps 400 counts. Neither one expands the data at all.

The last data issue: the curse of dimensionality

The curse of dimensionality → as the number of descriptive features grows, the volume of the feature space grows so fast that any realistic quantity of data becomes sparse within it, and reasoning based on closeness begins to fail.

Two calculations that make the curse concrete

Calculation one: how much of the space must a neighbourhood cover? To capture a fraction f of the volume of a unit cube in m dimensions, a smaller cube must have edge length

$$e = f^{1/m}$$

Taking $f = 0.01$, so a neighbourhood holding one percent of the data:

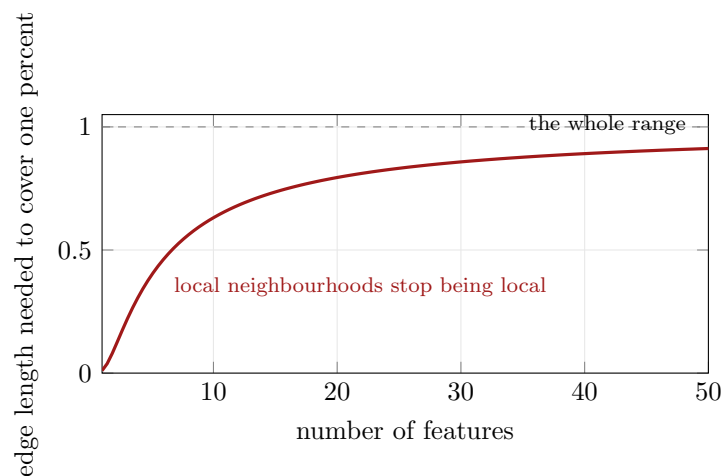
Features m	Edge length needed	Meaning
1	0.010	one percent of the range: genuinely local
2	0.100	ten percent of each range
5	0.398	forty percent of each range
10	0.631	sixty three percent of each range
20	0.794	almost eighty percent of each range
50	0.912	the neighbourhood is nearly the whole space

By fifty features, the nearest neighbours of a point are not near it in any meaningful sense. The word *neighbour* has quietly stopped meaning anything.

Calculation two: how thinly is the data spread? Divide every feature into ten bins. The number of cells in the space is 10^m .

Features m	Cells 10^m	Instances per cell with 1 000 000 rows
3	1 000	1 000
6	1 000 000	1
9	10^9	0.001
12	10^{12}	0.000001

With twelve features, a million rows leave 999 999 out of every million cells completely empty. Any question about a new instance is a question about an empty region.



- If there are many descriptive features, then a **large training dataset** is needed.
- Feature selection is important in any project and should generally be applied no matter what type of model is being developed.
- Some models are more susceptible than others:
 - **Similarity based approaches are particularly sensitive** and can struggle badly on data with many features, for exactly the reason the table above shows.
 - **Decision trees are more robust**, because a feature selection mechanism is built into the induction algorithm: at each split only the most informative feature is used and the rest are ignored.

In plain words

Adding a feature feels free, because it is just one more column. It is not free. Each new feature multiplies the space the data has to fill, and the data does not grow to match.

Putting It Together: Three Selection Scenarios

Each scenario below applies the two questions in order: does the approach fit the project, and does it fit the data?

Scenario one: a bank deciding on loan applications

Project requirements. Every refusal must be explained to the customer and to a regulator. Decisions are made in seconds, not milliseconds. The policy environment changes perhaps twice a year.

Data. Mostly categorical features, a categorical target, tens of thousands of rows.

Reasoning. Interpretability dominates, which rules out ensembles and support vector machines regardless of their accuracy. The categorical target and mostly categorical features suit information based and probability based models. Retraining twice a year is easy enough that a full rebuild is acceptable.

Shortlist: decision tree first, logistic regression as a comparison, naive Bayes as a baseline.

Scenario two: real time card fraud screening

Project requirements. Thousands of predictions per second. Fraud patterns change weekly. A missed fraud costs hundreds of times more than a false alarm. Explanations are useful for investigators but are not a legal requirement at the moment of the decision.

Data. Mostly continuous features, a categorical target, hundreds of millions of rows, and a heavily imbalanced target.

Reasoning. Prediction speed rules out nearest neighbour immediately. Frequent drift favours something that can be retrained often and cheaply. Imbalance means accuracy is worthless as a measure and the threshold must be set from the cost ratio, as computed earlier.

Shortlist: a gradient boosted ensemble for accuracy, logistic regression as a fast and easily retrained comparison, both evaluated with precision and recall rather than accuracy.

Scenario three: predicting equipment failure in a small factory

Project requirements. Only 300 labelled failures exist. Engineers have strong knowledge of which measurements influence which. Predictions are needed once an hour.

Data. A mixture of continuous sensor readings and categorical machine types, a categorical target, and a very small number of rows.

Reasoning. The dataset is far below the tipping point discussed earlier, so a generative model that can absorb the engineers' knowledge of dependencies is attractive. Speed is irrelevant at one prediction per hour. A powerful, low bias model would almost certainly overfit 300 rows.

Shortlist: a Bayesian network built with expert input, naive Bayes as a simpler comparison, and a shallow, heavily pruned decision tree.

Running the Comparison Honestly

Choosing a shortlist is only useful if the experiment that decides between the candidates is sound.

- **Set a baseline first.** Predict the most common level every time, or the mean value every time. If a sophisticated model cannot beat that, something is wrong. On Dataset A the baseline predicts default for everyone and is right five times out of eight, which is 62.5 percent. Any model scoring 62.5 percent has learned nothing.
- **Hold a test set aside and do not look at it.** Every glance followed by an adjustment leaks information and makes the final figure optimistic.
- **Tune on validation data.** With little data, use k fold cross validation: split into k parts, train on $k - 1$ and test on the one left out, and rotate.
- **Respect the structure when splitting.** Stratify when the target is imbalanced. Split by time when the data is time ordered, otherwise the model is allowed to see the future. Group by subject

when several rows describe the same person or machine.

- **Guard against leakage.** Any feature computed from information that will not exist at the moment of a real prediction will inflate the result and then fail in service. Fit any scaler or imputer on the training part only.
 - **Compare fairly.** Same splits, same preprocessing, same measure, and comparable tuning effort for each candidate.
 - **Report the spread, not just the average.** A mean across folds hides the difference between a stable model and a lucky one.
 - **Choose a measure that matches the goal.** Accuracy for a balanced target; precision and recall for an imbalanced one; mean absolute error or root mean squared error for regression, depending on how heavily large errors should count; and cost in currency wherever the cost of each error can be stated.
 - **Prefer the simpler model on a tie.** It costs less to run, less to explain, and less to maintain.
-

Where the Real Difficulty Lies

- In many ways the easy part of a project is building the models. The algorithms explain how to do that.
- What makes the work difficult, and also fascinating, is answering all the questions that surround the modelling.
- Throughout a project, intuition, experience and experimentation must be used to steer towards the best solution.

To help ensure a successful outcome, let the decisions be informed by:

1. becoming situationally fluent, so as to be able to converse with experts in the application domain;
2. exploring the data in order to understand it correctly;
3. spending time cleaning the data;
4. thinking hard about the best ways to represent the features;
5. and spending time designing the evaluation process correctly.

Mistakes that recur on real projects

- Optimising a target that was easy to label rather than the one the organisation cares about.
- Tuning on the test set, which is the commonest source of models that look excellent and then fail.
- Reporting high accuracy on an imbalanced dataset where a constant prediction would score similarly.
- Leaving a leaked feature in the data, such as a field that is only filled in after the outcome is known.
- Running a distance model on unscaled features, so one column silently dominates.
- Filling missing values without asking why they are missing.
- Reading a predictive relationship as a causal one. A model tells you what accompanies an outcome, not what would happen if you intervened.
- Handing over a model with nobody responsible for monitoring or retraining it.

Topics that naturally follow this material

- **Unsupervised learning**, where there is no target at all: clustering, dimensionality reduction, anomaly detection.
 - **Reinforcement learning**, where the signal is a delayed reward rather than a labelled answer.
 - **Deep learning**, where the representation of the data is itself learned rather than designed.
 - **Graphical models**, which make the dependency structure between features explicit.
 - **Multilabel classification**, where an instance carries several labels at once.
 - **Explainability techniques**, for models that are not interpretable by construction.
-

Revision Sheet

1. A model maps descriptive features to a target feature; training searches for that map using examples.
 2. The search is inductive: a general rule inferred from specific instances.
 3. First truth: an induced model is not guaranteed to be correct, because a rule true of a sample need not be true of the population.
 4. Therefore only performance on unseen data is evidence.
 5. Second truth: learning cannot occur unless the process is biased, because infinitely many rules fit any finite sample and they disagree everywhere else.
 6. That bias is the inductive bias: the assumptions defining the search space explored and the search process used.
 7. Restriction bias fixes what can be expressed; preference bias fixes what is chosen among the expressible.
 8. Strong bias risks underfitting; weak bias risks overfitting.
 9. Every project decision, from target definition to threshold setting, adds further bias, and each is settled by intuition, experience and experimentation. Hence the art.
 10. Four quantities carry four learning styles: entropy for questioning, distance for analogy, Bayes for belief, squared error for correction.
 11. Parametric means the model size does not grow with the data; nonparametric means it does.
 12. Generative models learn what each class looks like, then apply Bayes; discriminative models learn the boundary directly, and usually win in complex domains with plenty of data.
 13. No Free Lunch: no approach wins everywhere, because each encodes different assumptions, visible in the shape of its decision boundary.
 14. Selection depends on project requirements, namely speed, retraining capacity and interpretability, often more than on accuracy.
 15. Selection also depends on the data: the target type, the feature types, and the curse of dimensionality, with feature selection applied regardless of model type.
-

Glossary

Baseline

The simplest possible prediction rule, used as the score every real model must beat.

Black box

A model whose reasoning cannot readily be explained to a person.

Calibration

Whether predicted probabilities can be trusted as probabilities, not only as rankings.

Class conditional density

The distribution of the descriptive values within one target level.

Class posterior

The probability of a target level given the descriptive values.

Class prior

The probability of a target level before any evidence is considered.

Concept drift

A change over time in the relationship between features and target, which makes a model stale.

Curse of dimensionality

The exponential growth of the feature space with the feature count, which leaves data sparse.

Decision boundary

The surface in the feature space that separates regions predicted as different levels.

Descriptive features

The input columns describing each instance.

Discriminative model

A model that learns the boundary between classes.

Entropy

A measure of impurity or uncertainty in a collection, in bits.

Feature selection

Choosing an informative subset of the descriptive features.

Generalisation gap

The difference between performance on training data and on unseen data.

Generative model

A model that encodes the distribution of each class and could therefore produce similar data.

Gradient descent

Finding weights by repeatedly stepping downhill on the error surface.

Inductive bias

The assumptions defining the search space explored and the search process used.

Inductive learning

Inducing a general rule from a set of specific instances.

Information gain

The reduction in entropy achieved by splitting on a feature.

Instance

One row of the data; one example of the prediction subject.

Laplace smoothing

Adding one to every count so that no outcome is ever assigned probability zero.

Leakage

Accidental use of information that will not be available at real prediction time.

Learning rate

The size of each step taken in gradient descent.

No Free Lunch Theorem

The result that no approach outperforms all others across all problems.

Nonparametric model

A model whose parameter count grows with the number of instances.

Overfitting

Modelling noise and sample quirks instead of the underlying relationship.

Parametric model

A model whose representation size is independent of the number of instances.

Preference bias

The ordering an algorithm imposes over the rules that fit the data.

Prediction subject

The entity that one row represents.

Restriction bias

The limit on which rules an algorithm can express at all.

Situational fluency

Enough domain understanding to work credibly with experts in the application area.

Stale model

A deployed model whose accuracy has decayed because the domain changed.

Target feature

The column the model is trained to predict.

Underfitting

Failing to capture the real relationship because the model is too constrained.